



Testes avançados de JUnit 5

Neste módulo, vamos avançar para o universo dos **testes avançados de JUnit 5**, ampliando nossa capacidade de testar aplicações de forma mais completa e estratégica. Primeiro, abordaremos a **execução de testes**, entendendo como personalizar ciclos de execução, aplicar filtros e otimizar o processo para diferentes cenários de validação. Aprofundaremos também o estudo sobre **testes parametrizados**, onde aprenderemos a criar testes dinâmicos capazes de validar múltiplas entradas e saídas com menos código e mais eficiência. Em seguida, trataremos dos **testes com tratamento de exceções**, fundamentais para garantir que nossos sistemas lidem corretamente com situações inesperadas, evitando falhas silenciosas ou comportamentos indesejados. Vamos analisar como capturar e validar essas exceções utilizando os recursos que o JUnit 5 nos oferece. Finalizando o módulo, exploraremos os **testes de regressão**, focando em como assegurar que melhorias e correções não comprometam funcionalidades já existentes. Discutiremos ainda o papel da automação nesse contexto e como o JUnit pode ser integrado ao fluxo contínuo de desenvolvimento.

Execução de testes

Neste tema, vamos explorar como acontece a execução de testes no JUnit 5, entendendo as principais formas de rodar os testes, interpretar os resultados e integrar a execução ao nosso fluxo de desenvolvimento. Saber executar testes de maneira estruturada é um passo importante para tornar a prática de testes parte natural do nosso trabalho diário.

A execução de testes no JUnit 5 pode ser feita de diferentes maneiras, dependendo do ambiente de desenvolvimento e das ferramentas que estamos utilizando. Uma das formas mais comuns é executar os testes

diretamente pela IDE, como o IntelliJ IDEA, Eclipse ou Visual Studio Code. Nessas ferramentas, podemos rodar um único teste, uma classe inteira de testes ou todos os testes do projeto com poucos cliques. A visualização gráfica das execuções, com indicadores de sucesso e falha, nos ajuda a interpretar rapidamente os resultados.

Quando utilizamos ferramentas de automação como Maven ou Gradle, também podemos executar os testes através de comandos no terminal. Essa abordagem é especialmente útil quando integramos os testes em pipelines de integração contínua, garantindo que os testes sejam executados automaticamente a cada atualização do código. No Maven, por exemplo, o comando `mvn test` dispara a execução de todos os testes definidos no projeto, respeitando a configuração de diretórios e dependências estabelecidas.

Durante a execução dos testes, cada método anotado com `@Test` é chamado individualmente. Antes de cada teste, os métodos anotados com `@BeforeEach` são executados para preparar o ambiente, e após cada teste, os métodos anotados com `@AfterEach` são chamados para fazer a limpeza. Caso tenhamos métodos anotados com `@BeforeAll` e `@AfterAll`, eles são executados uma única vez antes e depois de toda a classe de testes, respectivamente. Esse ciclo de execução garante que cada teste seja isolado e que as condições de teste estejam controladas.

Ao analisarmos os resultados, é importante interpretar corretamente o que o relatório nos apresenta. Um teste que passa indica que as condições esperadas foram atendidas. Um teste que falha mostra geralmente a diferença entre o resultado esperado e o resultado obtido, ajudando a localizar rapidamente a origem do problema. Quando um teste apresenta erro, significa que uma exceção não prevista ocorreu durante a execução, o que pode indicar problemas como dados mal preparados, dependências ausentes ou falhas de ambiente.

Outra funcionalidade importante no JUnit 5 é a possibilidade de configurar **execuções condicionais**. Podemos, por exemplo, utilizar anotações como `@EnabledOnOs` ou `@DisabledOnJre` para definir testes que só devem ser

executados em determinados sistemas operacionais ou versões específicas do Java. Isso torna nossa suíte de testes mais flexível e adaptada a diferentes ambientes.

O JUnit 5 também permite o uso de **tags** para categorizar os testes. Com a anotação `@Tag`, podemos marcar testes com categorias específicas, como “integração”, “unitário” ou “lento”. Na execução, podemos filtrar quais categorias queremos rodar, o que é muito útil para acelerar as execuções durante o desenvolvimento, focando apenas nos testes mais relevantes para o momento.

Outro recurso interessante é a execução paralela de testes, que pode ser configurada para aproveitar melhor os recursos da máquina e reduzir o tempo total de execução, especialmente em projetos com uma abundância de casos de teste. O JUnit Platform oferece suporte a essa configuração, respeitando as dependências e a necessidade de isolamento entre os testes.

Executar testes de forma contínua e disciplinada nos ajuda a identificar rapidamente defeitos, a validar o impacto das mudanças no código e a garantir a evolução segura dos sistemas. Com a prática, a execução de testes se torna uma parte natural do ciclo de desenvolvimento, trazendo mais agilidade, segurança e confiança para as entregas.

Ao dominar as práticas de execução de testes que vimos neste tema, estamos prontos para seguir adiante, aprofundando nossa experiência com testes mais dinâmicos, parametrizados e voltados para cenários cada vez mais realistas. Nos próximos temas, vamos expandir ainda mais o uso do JUnit 5 para atender a essas novas demandas.

Testes parametrizados

Neste tema, vamos conhecer os **testes parametrizados**, uma funcionalidade do JUnit 5 que nos permite executar o mesmo teste com diferentes conjuntos de dados. Em vez de escrevermos vários métodos semelhantes somente para testar valores diferentes, podemos criar um único teste que varia automaticamente os parâmetros. Essa abordagem torna nossos

testes mais enxutos, claros e fáceis de manter, além de aumentar a cobertura de cenários de forma prática.

Ao trabalharmos com testes parametrizados, utilizamos a anotação `@ParameterizedTest` no lugar da tradicional `@Test`. Essa anotação indica que o método de teste será executado várias vezes, uma para cada conjunto de dados fornecido. Para definir os dados, o JUnit 5 oferece diferentes fontes, como `@ValueSource`, `@CsvSource`, `@MethodSource` e `@EnumSource`, entre outras. Cada uma dessas fontes nos permite alimentar o teste de maneira diferente, conforme a necessidade.

O `@ValueSource` é uma das formas mais simples de parametrizar um teste. Podemos usá-lo para fornecer uma lista de valores simples, como inteiros, strings ou booleans. Cada valor da lista será passado como argumento para o método de teste, que será executado individualmente para cada um deles. Esse recurso é ideal para validar comportamentos que dependem de entradas básicas, como verificações de formatos ou limites.

Quando precisamos testar combinações de múltiplos valores, podemos utilizar o `@CsvSource`. Ele permite definir linhas de valores separados por vírgulas, mapeados para os parâmetros do método de teste. Assim, conseguimos testar cenários mais complexos, como diferentes combinações de entradas e saídas esperadas, de forma bastante organizada. O `CsvSource` facilita ainda a leitura e a manutenção dos dados de teste.

O `@MethodSource` é usado quando queremos gerar os dados de teste programaticamente. Com ele, apontamos para um método que retorna um `Stream`, uma `Collection` ou um array de argumentos. Essa abordagem é útil para casos em que os dados de teste são mais elaborados, precisam ser calculados ou extraídos de fontes externas. Podemos criar métodos reutilizáveis para gerar dados dinamicamente e manter nossos testes flexíveis e adaptáveis.

Já o `@EnumSource` é ideal para testar todas ou algumas constantes de um enum específico. Em vez de escrevermos um teste para cada valor de enumeração, utilizamos o `EnumSource` para garantir que todas as

possibilidades sejam validadas automaticamente, economizando tempo e reduzindo o risco de esquecermos algum caso.

Ao escrevermos testes parametrizados, precisamos nos lembrar de que a clareza continua sendo essencial. Mesmo que o teste execute múltiplas vezes com dados diferentes, ele deve ser legível e fácil de entender. Podemos utilizar o `DisplayName` e o `DisplayNameGenerator` para personalizar como os testes são apresentados durante a execução, facilitando a interpretação dos resultados e a identificação de falhas específicas.

Os testes parametrizados também ajudam a tornar nossos testes mais completos. Em vez de limitar a validação a poucos casos, podemos testar facilmente múltiplos cenários, cobrindo extremos, valores inválidos, combinações improváveis e outros casos que poderiam passar despercebidos em testes manuais ou pouco variados.

Outro benefício dos testes parametrizados é que eles incentivam a organização dos casos de teste por comportamento, e não somente por dados. Em vez de criar um teste para cada conjunto de valores, mantemos o foco na lógica a ser validada, tornando o código de teste mais enxuto, expressivo e alinhado às boas práticas de desenvolvimento.

Dominar a criação de testes parametrizados é um passo importante para evoluirmos nossa abordagem de testes, principalmente em projetos que exigem validação de múltiplos cenários ou que lidam com grande diversidade de entradas. Ao utilizarmos essa funcionalidade de maneira consciente, ganhamos produtividade e fortalecemos a qualidade dos sistemas que desenvolvemos.

Nos próximos temas, vamos continuar expandindo nossa experiência com testes mais avançados no JUnit 5, aprendendo a lidar com situações ainda mais desafiadoras e consolidando as boas práticas que já começamos a construir até aqui.

Testes com tratamento de exceções

Neste tema, vamos entender como trabalhar com **testes com tratamento de**

exceções no JUnit 5. Testar exceções é uma parte fundamental da validação de sistemas robustos, pois precisamos garantir que o software reaja corretamente a situações inesperadas, como entradas inválidas, operações ilegais ou falhas de comunicação. Ao dominar o tratamento de exceções nos testes, conseguimos verificar não apenas se o sistema funciona quando tudo está certo, mas também se ele se comporta adequadamente diante de problemas.

No JUnit 5, a principal forma de testar o lançamento de exceções é utilizando o método `assertThrows`. Com ele, podemos executar um trecho de código que esperamos que gere uma exceção específica e verificar se a exceção foi realmente lançada. Esse método exige que passemos dois parâmetros: o tipo da exceção esperada e a execução de um bloco de código que deve provocar essa exceção. Se o código não lançar a exceção esperada, o teste falha automaticamente.

O uso do `assertThrows` nos permite validar não apenas que uma exceção ocorreu, mas também inspecionar o objeto da exceção capturada. Isso é muito útil para verificar mensagens de erro específicas, códigos de status ou qualquer outro detalhe importante da exceção. Dessa forma, conseguimos criar testes mais precisos e garantir que o sistema forneça informações úteis quando um erro acontece.

Além do `assertThrows`, também podemos utilizar o `assertDoesNotThrow` em cenários onde queremos confirmar que determinado bloco de código não lança nenhuma exceção inesperada. Essa abordagem é importante para validar fluxos críticos do sistema que precisam ser estáveis e livres de erros em condições normais.

Ao estruturarmos testes com tratamento de exceções, devemos sempre pensar na clareza e na especificidade. Precisamos garantir que o teste esteja realmente verificando o comportamento correto e não somente capturando qualquer tipo de erro. Por isso, é recomendável testar exceções específicas, e não exceções genéricas, sempre que possível. Isso torna nossos testes mais confiáveis e resistentes a mudanças futuras no código.

Em alguns casos, podemos querer testar múltiplos cenários onde diferentes

exceções podem ocorrer, dependendo das condições de entrada. Para essas situações, podemos escrever múltiplos métodos de teste, cada um validando um cenário específico. Essa prática mantém os testes focados e facilita a identificação de problemas quando algo dá errado.

Outro recurso útil do JUnit 5 é a possibilidade de combinar o `assertThrows` com outras assertivas. Por exemplo, podemos capturar a exceção e, em seguida, utilizar `assertEquals` para verificar a mensagem de erro associada. Isso torna nossos testes mais completos e garante que a comunicação de erro do sistema esteja adequada às expectativas dos usuários ou das integrações.

Testar o tratamento de exceções também nos ajuda a identificar pontos do sistema que podem precisar de melhorias em termos de estabilidade e comunicação de falhas. Quando encontramos dificuldade para provocar a exceção em um teste, isso pode indicar que o código precisa ser melhorado para lidar de maneira mais clara com situações inesperadas.

Ao incluirmos o tratamento de exceções de forma sistemática nos nossos testes, construímos aplicações mais confiáveis, seguras e preparadas para lidar com a variedade de problemas que podem surgir em ambientes reais. Além disso, transmitimos uma mensagem de responsabilidade e maturidade técnica ao garantir que não estamos apenas validando o sucesso dos nossos sistemas, mas também sua capacidade de reagir corretamente ao erro.

Com o que aprendemos neste tema, estaremos mais preparados para enfrentar os desafios de criar sistemas resilientes e de alta qualidade. Nos próximos temas, vamos continuar aprimorando nossa prática de testes, abordando novas técnicas e cenários que nos permitirão evoluir ainda mais como desenvolvedores de software focados na excelência.

Testes de regressão

Neste tema, vamos entender o papel dos **testes de regressão** em uma estratégia de qualidade contínua. À medida que o software evolui, novas funcionalidades são adicionadas, melhorias são implementadas e correções

de defeitos são realizadas. No entanto, cada alteração no código pode introduzir riscos que afetam partes que já estavam funcionando corretamente. É justamente para controlar esses riscos que utilizamos testes de regressão.

Testes de regressão são aqueles que revalidam comportamentos já conhecidos do sistema após alguma modificação no código. Nosso objetivo com esses testes é garantir que mudanças recentes não tenham causado efeitos colaterais indesejados em funcionalidades existentes. Quando estruturamos uma boa suíte de testes de regressão, conseguimos evoluir o sistema com maior segurança e confiança.

No JUnit 5, os testes de regressão seguem a mesma estrutura dos testes unitários tradicionais. A diferença está no seu propósito. Enquanto um novo teste geralmente busca validar uma nova funcionalidade, um teste de regressão confirma que funcionalidades antigas continuam se comportando como esperado. Por isso, é fundamental mantermos nossos testes atualizados conforme o sistema evolui, incorporando novos cenários e adaptando os existentes quando necessário.

Uma prática comum para fortalecer a cobertura de regressão é transformar defeitos corrigidos em novos casos de teste. Sempre que um bug é identificado e corrigido, criamos um teste que reproduz o problema e valida a solução implementada. Esse teste passa a fazer parte da suíte de regressão e ajuda a evitar que o mesmo erro retorne no futuro.

Os testes de regressão são especialmente relevantes em projetos de médio e grande porte, onde o volume de código e a complexidade das interações aumentam ao longo do tempo. Em ambientes de desenvolvimento ágil, onde entregas frequentes fazem parte do processo, a automação dos testes de regressão se torna ainda mais necessária para garantir que o ritmo de evolução não comprometa a qualidade do produto.

No contexto do JUnit 5, podemos organizar os testes de regressão em classes específicas, agrupando-os conforme os módulos ou funcionalidades do sistema. Também podemos utilizar recursos como @Tag para identificar testes de regressão e facilitar sua execução isolada,

especialmente em pipelines de integração contínua, onde a execução seletiva de testes pode otimizar o tempo de entrega.

Outro ponto importante é que a manutenção da suíte de testes de regressão deve ser encarada como parte integrante do desenvolvimento. À medida que requisitos mudam ou funcionalidades são descontinuadas, precisamos revisar e ajustar os testes de regressão correspondentes. Manter testes desatualizados ou irrelevantes gera ruído nas execuções e pode comprometer a confiança nos resultados.

Uma boa prática para melhorar a efetividade dos testes de regressão é revisar periodicamente sua cobertura, garantindo que os principais fluxos de negócio, as integrações críticas e os pontos de maior risco estejam devidamente testados. Para apoiar essa análise, podemos utilizar ferramentas de cobertura de código, que indicam quais trechos do sistema estão sendo efetivamente verificados pelos testes.

Ao incorporar os testes de regressão como parte fundamental da nossa rotina de desenvolvimento, transformamos a evolução do software em um processo mais seguro e sustentável. Reduzimos a incidência de falhas em produção, aumentamos a previsibilidade das entregas e fortalecemos a confiança dos usuários nas soluções que construímos.

Com a compreensão sólida que desenvolvemos neste tema, estamos mais preparados para estruturar e manter suítes de testes que acompanhem o crescimento dos projetos, garantindo qualidade não somente no presente, mas também no futuro do software que desenvolvemos. Nos próximos temas, continuaremos explorando técnicas que aprofundam ainda mais nossa capacidade de criar sistemas de alta confiabilidade.

Conteúdo Bônus

Recomendamos a leitura do livro *Mastering Software Testing with JUnit 5*, de Boni García, como material complementar para este módulo. A obra oferece uma abordagem prática e aprofundada sobre o framework JUnit 5, abordando desde os fundamentos da qualidade de software até técnicas avançadas de testes. Explora o modelo de programação Jupiter, recursos

como testes dinâmicos, injeção de dependências, execução paralela e integração com ferramentas como Mockito, Spring, Selenium, Cucumber e Docker. Além disso, apresenta boas práticas para escrever testes significativos e gerenciar atividades de teste em projetos de software em constante evolução. Com exemplos reais e orientações práticas, o livro capacita desenvolvedores e testadores a aprimorar a qualidade de suas aplicações Java por meio de testes eficazes e bem estruturados.

Referências Bibliográficas

GARCÍA, Boni. ***Mastering Software Testing with JUnit 5: Comprehensive guide to develop high quality Java applications***. Birmingham: Packt Publishing, 2017.

Ir para exercício